

BÀI TẬP THỰC HÀNH

BÀI 1: Class `Book`

Đề bài: Xây dựng class `Book` mô phỏng một cuốn sách trong thư viện.

Yêu cầu cụ thể:

1. Khai báo các thuộc tính (chưa cần `private`): `title (String)`, `author (String)`, `year (int)`, `price (double)`
2. Viết 2 constructor:
 - o Constructor mặc định: gán `title = "Unknown"`, `author = "Unknown"`, `year = 2000`, `price = 0`
 - o Constructor có 4 tham số: nhận đầy đủ thông tin, sử dụng `this` để gán giá trị
3. Viết phương thức `displayInfo()` in ra toàn bộ thông tin sách theo định dạng:
4. Sách: `[title]` - Tác giả: `[author]` - Năm: `[year]` - Giá: `[price]`
5. Viết class `Main` để test: tạo 2 object, một bằng constructor mặc định, một bằng constructor có tham số. Gọi `displayInfo()` cho cả 2.

Phần mở rộng (điểm cộng):

- Thêm constructor thứ 3 chỉ nhận `title` và `author`, năm mặc định là năm hiện tại (2026), giá mặc định là 100000.
- Thêm phương thức `applyDiscount(double percent)` để giảm giá sách theo phần trăm (vd: `applyDiscount(10)` giảm 10%).

BÀI 2: Class `Rectangle`

Đề bài: Xây dựng class `Rectangle` mô phỏng hình chữ nhật, áp dụng constructor overloading.

Yêu cầu cụ thể:

1. Khai báo 2 thuộc tính: `width (double)`, `height (double)`
2. Viết 3 constructor (overloading):
 - o Không tham số: tạo hình vuông cạnh 1 (`width = 1`, `height = 1`)
 - o 1 tham số `side`: tạo hình vuông cạnh `side` (`width = side`, `height = side`)
 - o 2 tham số `width`, `height`: tạo hình chữ nhật bình thường (dùng `this`)
3. Viết các phương thức:
 - o `getArea()`: trả về diện tích
 - o `getPerimeter()`: trả về chu vi
 - o `isSquare()`: trả về `true` nếu là hình vuông, ngược lại `false`
4. Viết `main` test cả 3 cách tạo object, in diện tích, chu vi, và kiểm tra có phải hình vuông không.

Phần mở rộng (điểm cộng):

- Thêm constructor thứ 4 nhận vào 1 object `Rectangle` khác để tạo bản sao (copy constructor).
- Thêm phương thức `scale(double factor)` để phóng to/thu nhỏ hình chữ nhật theo hệ số.
- Suy nghĩ: tại sao không thể có 2 constructor cùng nhận 1 tham số `double`? Hãy thử và giải thích lỗi.

BÀI 3: Class `BankAccount`

Đề bài: Xây dựng class `BankAccount` mô phỏng tài khoản ngân hàng.

Yêu cầu cụ thể:

1. Khai báo các thuộc tính `private`: (tự đặt tên thuộc tính)
 - số tài khoản
 - tên chủ tài khoản
 - số dư
2. Viết constructor có 3 tham số. Trong constructor:
 - Nếu số dư < 0 thì gán số dư = 0 và in cảnh báo
 - Các trường còn lại gán bình thường, dùng `this`
3. Viết getter cho cả 3 thuộc tính.
4. Viết setter có validation:
 - Tên chủ tài khoản: chỉ chấp nhận tên không rỗng (`!= null && !name.trim().isEmpty()`)
 - **KHÔNG** viết setter cho `accountNumber` (số tài khoản không được đổi)
 - **KHÔNG** viết setter trực tiếp cho số dư
5. Thay vì setter cho số dư, viết 2 phương thức:
 - `deposit(double amount)`: nạp tiền. Chỉ chấp nhận `amount > 0`
 - `withdraw(double amount)`: rút tiền. Chỉ chấp nhận `amount > 0` và `amount <= balance`
6. Viết phương thức `displayInfo()` in ra thông tin (không in số tài khoản đầy đủ, chỉ in 4 ký tự cuối, ví dụ ****1234).
7. Viết `main`: tạo 1 tài khoản, thử nạp/rút tiền, thử các trường hợp lỗi (nạp số âm, rút quá số dư).

Phản mở rộng (điểm cộng):

- Thêm phương thức `transfer(BankAccount other, double amount)` để chuyển tiền sang tài khoản khác.
- Suy nghĩ: tại sao số tài khoản không nên có setter? Trả lời bằng comment trong code.

BÀI 4: Class `Employee`

Đề bài: Xây dựng class `Employee` quản lý nhân viên trong công ty, sử dụng static để theo dõi dữ liệu chung.

Yêu cầu cụ thể:

1. Khai báo thuộc tính:
 - o `private int id` - mã nhân viên (tự động tăng, không nhận từ tham số)
 - o `private String name`
 - o `private double salary`
 - o `private static int employeeCount = 0` - đếm số nhân viên hiện tại
 - o `private static int nextId = 1000` - id sẽ cấp tiếp theo
 - o `public static String companyName = "TechCorp"` - tên công ty
 - o `private static double totalSalary = 0` - tổng lương cả công ty
2. Viết constructor nhận `name` và `salary`. Trong constructor:
 - o Tự động gán `id = nextId`, sau đó tăng `nextId` lên 1
 - o Tăng `employeeCount` lên 1
 - o Cộng `salary` vào `totalSalary`
3. Viết getter cho `id`, `name`, `salary`. Setter chỉ cho `name` và `salary`.
 - o Lưu ý: khi setter cho `salary` thay đổi giá trị, phải cập nhật lại `totalSalary` cho đúng.
4. Viết các phương thức static:
 - o `getEmployeeCount()`: trả về số nhân viên
 - o `getTotalSalary()`: trả về tổng lương
 - o `getAverageSalary()`: trả về lương trung bình (cẩn thận chia cho 0)
5. Viết main: tạo ít nhất 3 nhân viên, in id của từng người (phải là 1000, 1001, 1002), in tổng số nhân viên, tổng lương, lương trung bình. Thay đổi lương một nhân viên và kiểm tra `totalSalary` có cập nhật đúng không.

Phần mở rộng (điểm cộng):

- Thêm phương thức static `changeCompanyName(String newName)` để đổi tên công ty.
- Suy nghĩ và trả lời bằng comment: nếu trong main ta không tạo object nào, gọi `Employee.getAverageSalary()` có chạy được không? Tại sao?
- Thêm phương thức instance `raiseSalary(double percent)` để tăng lương theo phần trăm, vẫn đảm bảo `totalSalary` chính xác.

BÀI 5: Class `Product` + Package

Đề bài: Xây dựng hệ thống quản lý sản phẩm của một cửa hàng, tổ chức theo package.

Cấu trúc thư mục bắt buộc:

```
src/
├── model/
│   └── Product.java
├── util/
│   └── ProductValidator.java
└── Main.java
```

Yêu cầu cụ thể:

Class `Product` (package `model`):

- Thuộc tính `private`:
 - `productCode` (String) - mã sản phẩm, định dạng "P-XXXX" với XXXX là số tự động tăng từ 0001
 - `name` (String)
 - `price` (double)
 - `quantity` (int) - số lượng tồn kho
 - `static int counter = 1` - dùng để tạo mã sản phẩm
 - `static int totalProducts = 0` - tổng số sản phẩm đã tạo
 - `static double totalRevenue = 0` - tổng doanh thu của cửa hàng
- Viết 3 constructor (overloading):
 - Không tham số: tạo sản phẩm với `name = "Unknown"`, `price = 0`, `quantity = 0`
 - 2 tham số (`name`, `price`): `quantity` mặc định = 0
 - 3 tham số (`name`, `price`, `quantity`): đầy đủ
 - Tất cả constructor đều phải tự sinh `productCode` theo định dạng P-0001, P-0002,... (gợi ý: dùng `String.format("P-%04d", counter)`)
- Getter cho tất cả thuộc tính. Setter chỉ cho `name`, `price`, `quantity` (có validation: `price >= 0`, `quantity >= 0`, `name` không rỗng).
 - KHÔNG** có setter cho `productCode`.
- Phương thức instance:
 - `sell(int amount)`: bán sản phẩm. Kiểm tra `amount > 0` và `amount <= quantity`. Nếu thỏa, giảm `quantity` và cộng `amount * price` vào `totalRevenue`.
 - `restock(int amount)`: nhập thêm hàng. Kiểm tra `amount > 0`.
 - `displayInfo()`: in thông tin sản phẩm.
- Phương thức static:
 - `getTotalProducts()`, `getTotalRevenue()`
 - `getStoreReport()`: trả về chuỗi báo cáo cửa hàng (tổng số sản phẩm, tổng doanh thu)

Class `ProductValidator` (package `util`):

- Viết class chứa các phương thức static để kiểm tra dữ liệu:
 - `isValidName(String name)`: tên không null, không rỗng, độ dài ≥ 2
 - `isValidPrice(double price)`: `price >= 0`
 - `isValidQuantity(int quantity)`: `quantity >= 0`
 - Class này **KHÔNG** có thuộc tính nào, chỉ có phương thức static.
- Cập nhật class `Product`: trong setter và constructor, sử dụng `ProductValidator` thay vì viết logic kiểm tra trực tiếp.

Class `Main` (không package):

1. Import đầy đủ. Tạo ít nhất **4 sản phẩm** bằng các constructor khác nhau, bán/nhập hàng, in báo cáo cửa hàng cuối cùng, kiểm tra mã sản phẩm tự động đúng định dạng.

Phần mở rộng (điểm cộng):

- Thêm phương thức instance `applyPromotion(double discountPercent)` và phương thức static `applyGlobalPromotion(Product[] products, double discountPercent)` áp dụng giảm giá cho mảng sản phẩm.
- Suy nghĩ: nếu một sản phẩm bị “hủy” (không bán nữa), `totalProducts` có nên giảm không? Bạn xử lý thế nào? Hãy thêm phương thức `discontinue()` và giải thích lựa chọn của bạn bằng comment.
- Tự thiết kế thêm class `Category` ở package `model` để phân loại sản phẩm (sản phẩm có thuộc loại nào). Class `Product` có thêm thuộc tính `category` kiểu `Category`.